

Smoothing - Examples in R

B.A. Hartlaub and A.S. Wagaman

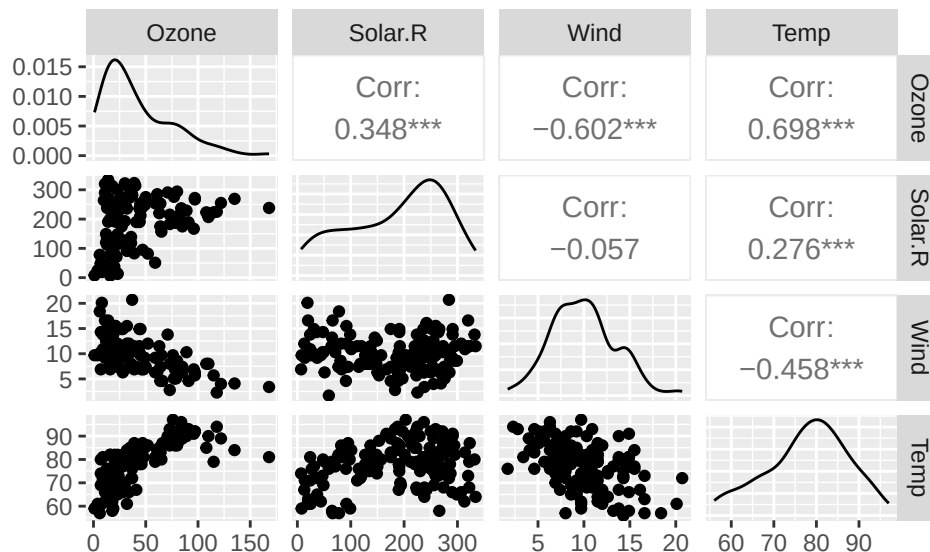
We note that some plots may be able to be updated with ggplot2, this may be revisited in the future.

Smoothing - Nonparametric Regression

The text presents three different linear smoothers (one per section) for us to consider.

To illustrate all three, we will use the `airquality` data set in R. We can only consider relationships between quantitative variables from what is presented in the text, so we'll remove the categorical variables from consideration.

```
data(airquality)
airquality <- select(airquality, -Month, -Day)
ggpairs(airquality)
```

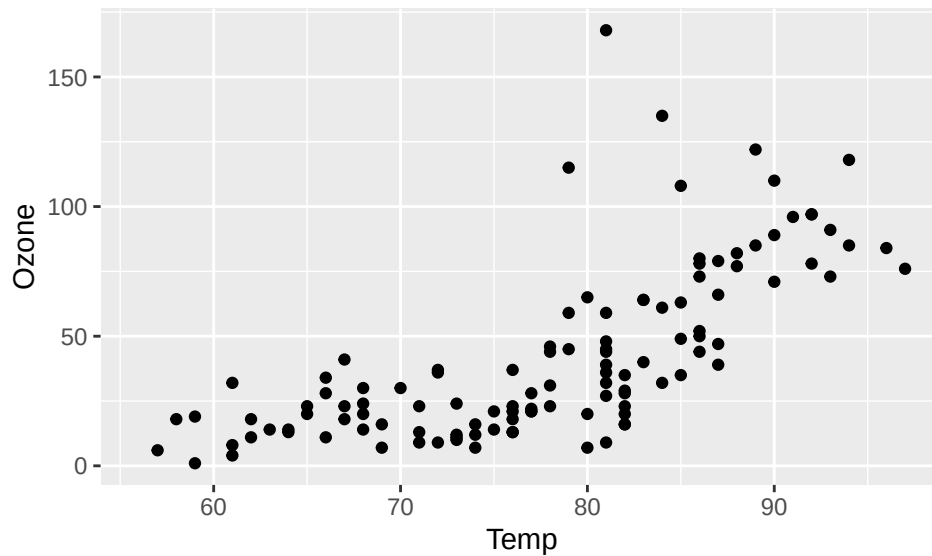


The relationship between Ozone and Temp looks interesting, possibly non-linear, so let's focus on that.

Local Averaging (Hollander, Wolfe, and Chicken 14.1)

```
gf_point(Ozone ~ Temp, data = airquality)
```

```
## Warning: Removed 37 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

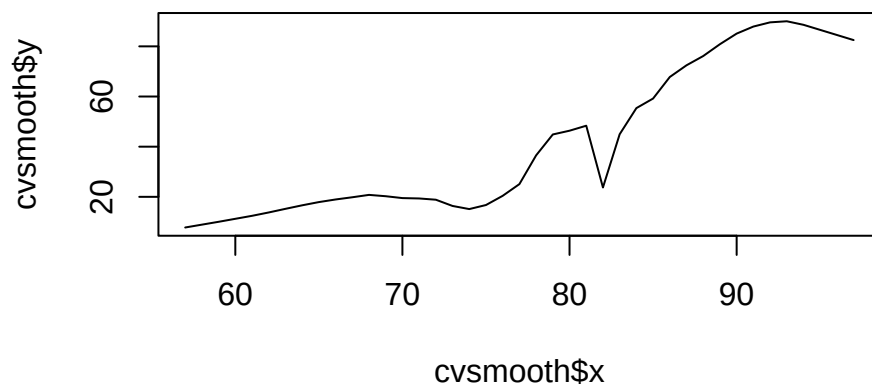


To apply local averaging, we use the `supsmu` function (in `stats` package) as follows:

```
cvsmooth <- with(airquality, (supsmu(x = Temp, y = Ozone, bass = 0, span = "cv")))

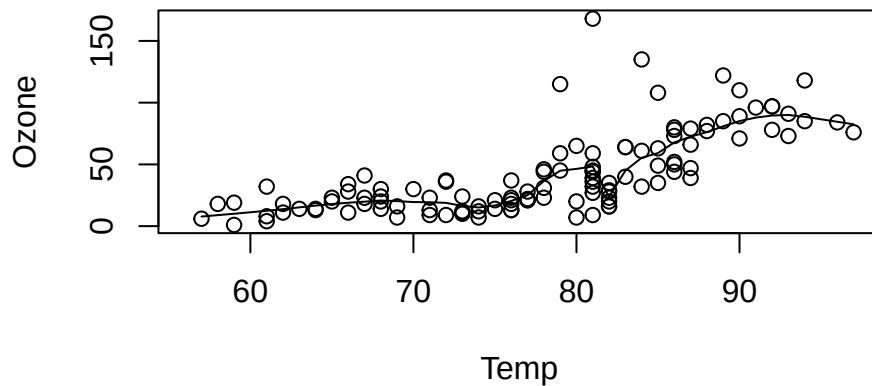
## Warning in supsmu(x = Temp, y = Ozone, bass = 0, span = "cv"): 37 observations
## with NAs, NaNs and/or Infs deleted

plot(cvsmooth, type = "l")
```



How can we plot this with our data? This uses base R graphics (will show `ggplot2` examples for loess smoothers).

```
with(airquality, plot(Temp, Ozone))
lines(cvsmooth)
```

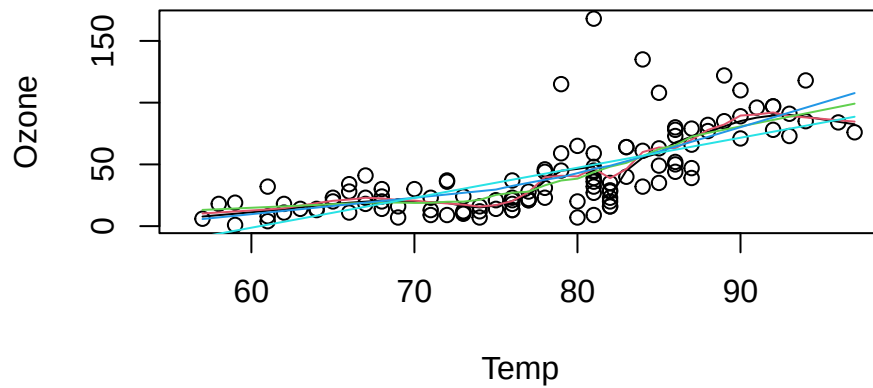


Let's try out some different spans, other than the default. From the help menu, span is described as the fraction of the observations in the span of the running lines smoother, or "cv" to choose this by leave-one-out cross-validation (LOOCV). So, we need to specify a value between 0 and 1 for span to see how this impacts the estimated function. This is different than the number of observations because it has been scaled, but you can map back and forth.

```
smoothcv <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = "cv"))) # default
smooth1 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = 0.1)))
smooth2 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = 0.3)))
smooth3 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = 0.7)))
# should be oversmoothed
smooth4 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = 1)))
```

Now we plot the results:

```
with(airquality, plot(Temp, Ozone)) # original data points
lines(smoothcv) # black
lines(smooth1, col = 2) # red
lines(smooth2, col = 3) # green
lines(smooth3, col = 4) # blue
lines(smooth4, col = 5) # light blue
```

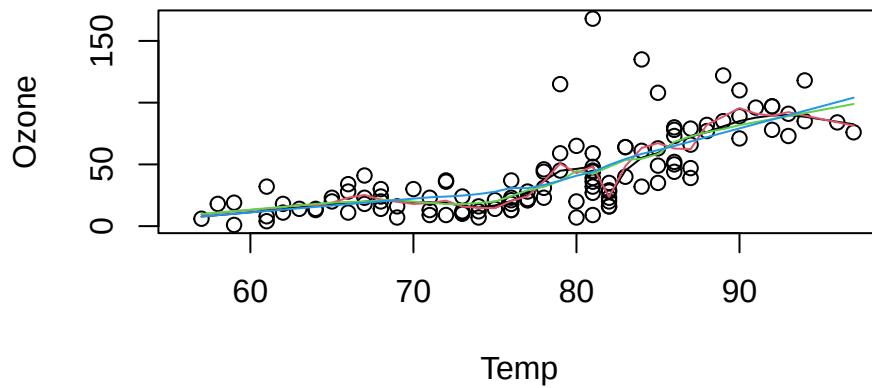


We could also do this with the tweeter, midrange, and woofer spans applied to the entire set of Xs (instead of being used in the variable span computation).

```
smoothcv <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = "cv"))) # default
smooth1 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = 0.05))) # tweeter
smooth2 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = 0.2))) # midrange
smooth3 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = 0.5))) # woofer
```

Now we plot the results:

```
with(airquality, plot(Temp, Ozone)) # points only
lines(smoothcv) # black
lines(smooth1, col = 2) # red
lines(smooth2, col = 3) # green
lines(smooth3, col = 4) # blue
```

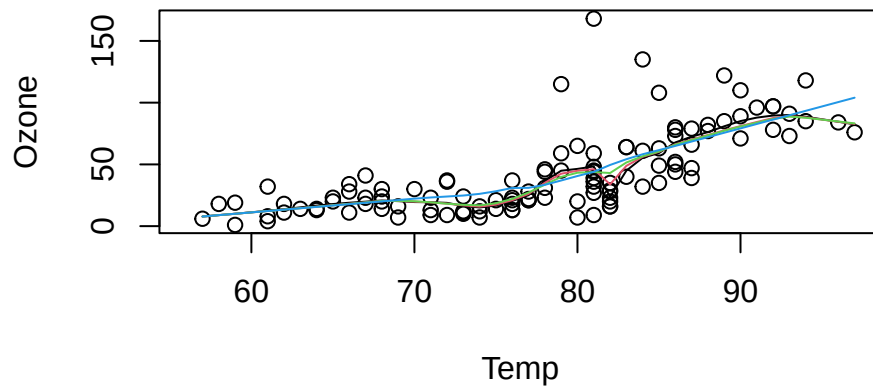


How does the `bass` parameter fit into this? The parameter is 0 by default (so you could leave it out of the function call). Values go from 0 to 10, and as they increase, you get smoother curves.

```
smoothcv <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 0, span = "cv"))) # default
smooth1 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 2, span = "cv")))
smooth2 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 5, span = "cv")))
smooth3 <- with(airquality,
  (supsmu(x = Temp, y = Ozone, bass = 10, span = "cv")))
```

Now we plot the results:

```
with(airquality, plot(Temp, Ozone))
lines(smoothcv) # black
lines(smooth1, col = 2) # red
lines(smooth2, col = 3) # green
lines(smooth3, col = 4) # blue
```



What is IN the supsmu objects? Let's look at one.

```
smooth3
```

```
## $x
## [1] 57 58 59 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82
## [26] 83 84 85 86 87 88 89 90 91 92 93 94 96 97
##
## $y
## [1] 7.802497 8.914441 10.026386 12.250275 13.362220 14.474165
## [7] 15.586110 16.698054 17.809999 18.921944 20.033888 21.145833
## [13] 22.257778 23.216249 23.997668 24.781634 26.154982 28.274956
## [19] 30.900955 31.040587 33.445627 36.967950 40.599969 43.971790
## [25] 49.545825 54.199665 58.220473 61.579276 64.965663 68.516493
## [31] 72.067324 75.618155 79.168986 82.719816 86.270647 89.821478
## [37] 93.372309 100.473970 104.024801
```

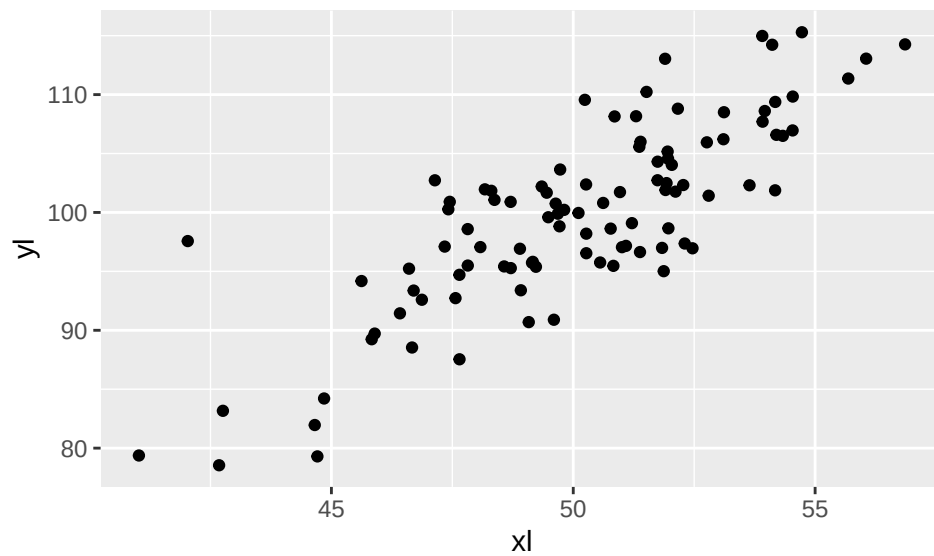
```
unique(sort(airquality$Temp))
```

```
## [1] 56 57 58 59 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81
## [26] 82 83 84 85 86 87 88 89 90 91 92 93 94 96 97
```

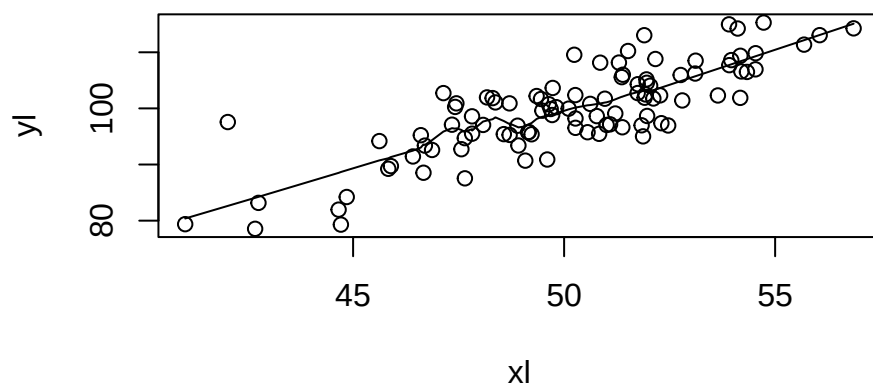
Basically, the X values where the smoother was evaluated are listed - these should be the X values in the data set (here we miss one at a tail end). Then, we get the y's corresponding to them after applying the smoother. You won't be able to see the cv chosen span.

Toy Data Example

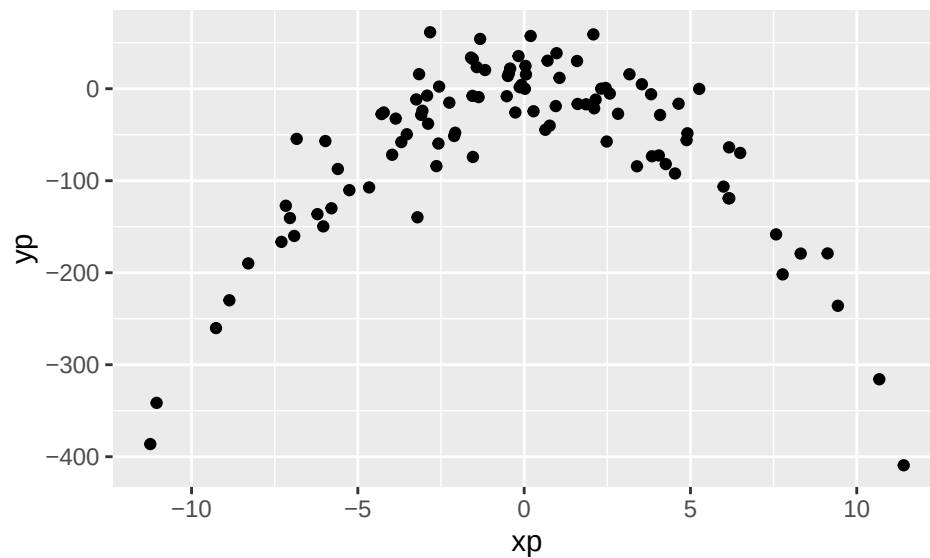
```
# basic linear pattern
set.seed(42)
x1 <- rnorm(100, 50, 3)
y1 <- 2*x1 + rnorm(100, 0, 5)
gf_point(y1 ~ x1)
```



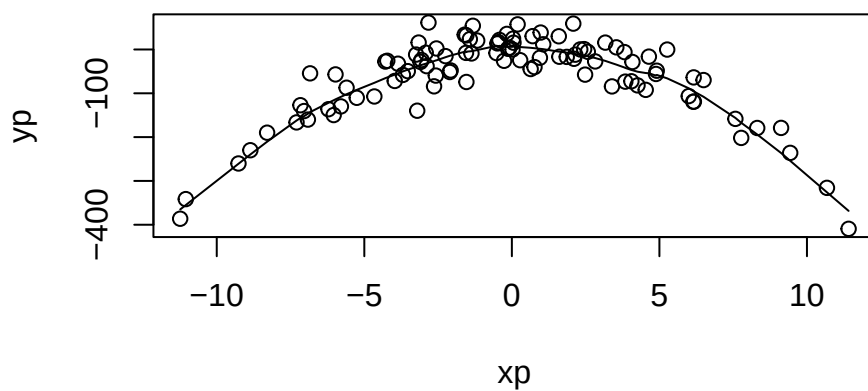
```
smoothcv <- supsmu(x = x1, y = y1, bass = 0, span = "cv") # default
plot(x1, y1)
lines(smoothcv) # black
```



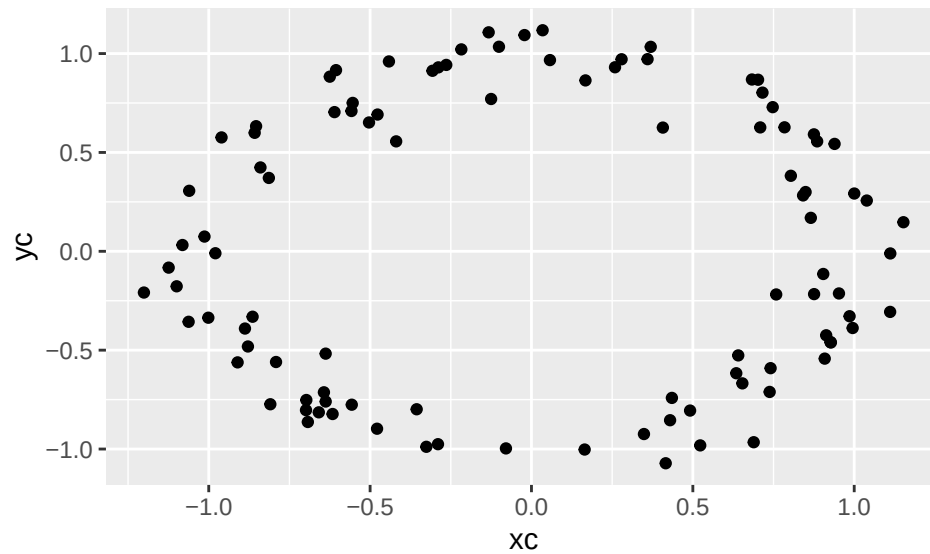
```
# parabola
set.seed(58)
xp <- rnorm(100, 0, 5)
yp <- (-3)*(xp^2) + 2*xp + rnorm(100, 0, 40)
gf_point(yp ~ xp)
```



```
smoothcv <- supsmu(x = xp, y = yp, bass = 0, span = "cv") # default
plot(xp, yp)
lines(smoothcv) # black
```

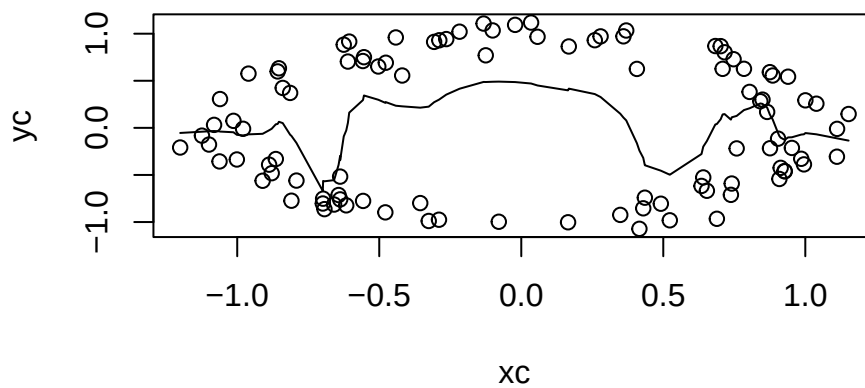


```
# circle - don't expect it to do well
set.seed(85)
angle <- runif(100, -2*pi, 2*pi)
xc <- sin(angle) + rnorm(100, 0, 0.1)
yc <- cos(angle) + rnorm(100, 0, 0.1)
gf_point(yc ~ xc)
```

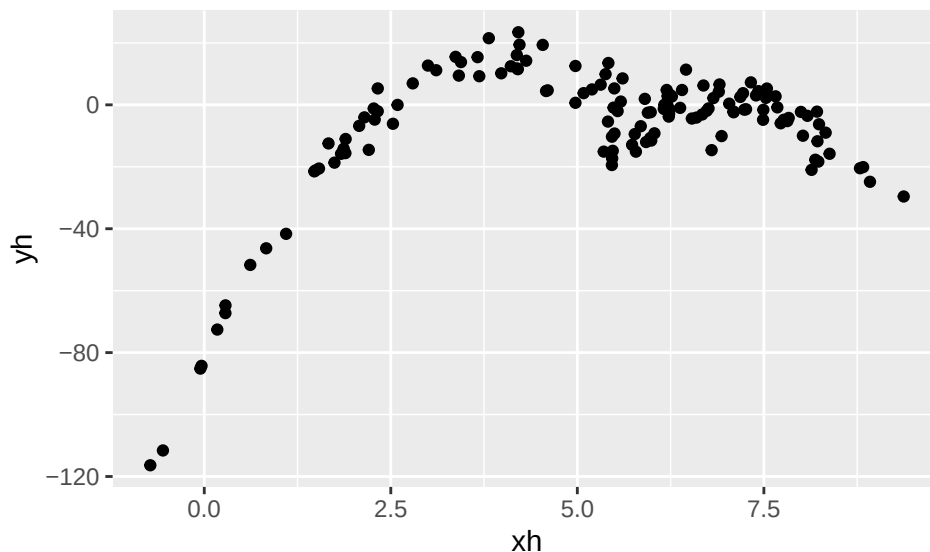



```
smoothcv <- supsmu(x = xc, y = yc, bass = 0, span = "cv") # default
plot(xc, yc, main = "Junk Fit because of Failed Vertical Line Test!")
lines(smoothcv) # black
```

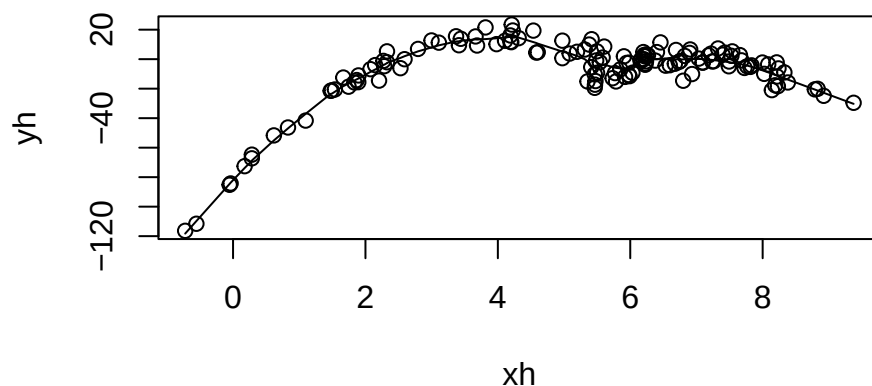
Junk Fit because of Failed Vertical Line Test!



```
# two humps
set.seed(89)
xh1 <- rnorm(75, 4, 2)
xh2 <- rnorm(75, 7, 1)
yh1 <- 15-6*(xh1-4)^2 + rnorm(75, 0, 5)
yh2 <- -6*(xh2-7)^2 + rnorm(75, 0, 5)
xh <- c(xh1[xh1<6], xh2[xh2>5])
yh <- c(yh1[xh1<6], yh2[xh2>5])
gf_point(yh ~ xh)
```



```
smoothcv <- supsmu(x = xh, y = yh, bass = 0, span = "cv") # default
plot(xh, yh)
lines(smoothcv) # black
```



Local Regression (Hollander, Wolfe, and Chicken 14.2)

The R commands associated with fitting LOESS models are `loess` and `loess.as`. The first function is in base R but the second is in the `fANCOVA` library (loaded above).

Our text states that the `loess` command requires data sorted by X values, so we can use the `arrange` command (or the `sort` command, or several other options) to do this.

```
airquality2 <- arrange(airquality, Temp)
```

We specify that we want locally linear regression with `degree = 1` (meaning a linear regression), and we can set a span when we call the command. The span value is α , the proportion of values of the sample we want used in each local regression. If you set `degree 2`, it will use quadratic regression.

```
lrfit <- loess(Ozone ~ Temp, data = airquality2, degree = 1, span = 0.5)
names(lrfit)
```

```
## [1] "n"          "fitted"      "residuals"  "enp"        "s"          "one.delta"
## [7] "two.delta"  "trace.hat"   "divisor"    "robust"      "pars"       "kd"
## [13] "call"       "terms"      "xnames"     "x"          "y"          "weights"
## [19] "na.action"
```

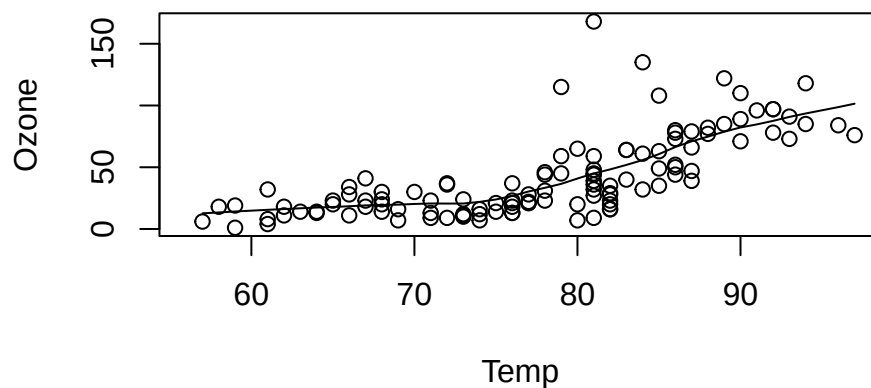
The fitted object has a number of pieces to it. The fitted values are found as:

```
lrfit$fitted
```

```
## [1] 12.66665 13.38565 14.16427 14.16427 15.53586 15.53586 15.53586
## [8] 16.12526 16.12526 16.76687 17.36913 17.36913 17.84048 17.84048
## [15] 18.42152 18.42152 18.42152 19.10895 19.10895 19.10895 19.44539
## [22] 19.44539 19.44539 19.44539 19.71652 19.71652 20.25422 20.67039
## [29] 20.67039 20.67039 20.58039 20.58039 20.58039 20.62650 20.62650
## [36] 20.62650 20.62650 21.96099 21.96099 21.96099 23.90578 23.90578
## [43] 26.76084 26.76084 26.76084 26.76084 26.76084 26.76084 30.35771
## [50] 30.35771 30.35771 30.35771 33.33303 33.33303 33.33303 33.33303
## [57] 36.51377 36.51377 36.51377 40.86243 40.86243 40.86243 44.95873
## [64] 44.95873 44.95873 44.95873 44.95873 44.95873 44.95873 44.95873
## [71] 44.95873 44.95873 48.24260 48.24260 48.24260 48.24260 48.24260
## [78] 48.24260 48.24260 51.88190 51.88190 51.88190 55.82710 55.82710
## [85] 55.82710 60.65124 60.65124 60.65124 60.65124 66.35884 66.35884
## [92] 66.35884 66.35884 66.35884 66.35884 70.99005 70.99005 70.99005
## [99] 70.99005 74.78487 74.78487 78.75247 78.75247 82.20729 82.20729
## [106] 82.20729 84.68874 87.54349 87.54349 87.54349 90.75618 90.75618
## [113] 93.54986 93.54986 98.69688 101.45857
```

We can plot this with our data as:

```
with(airquality2, plot(Temp, Ozone))
lines(lrfit$x, lrfit$fitted)
```

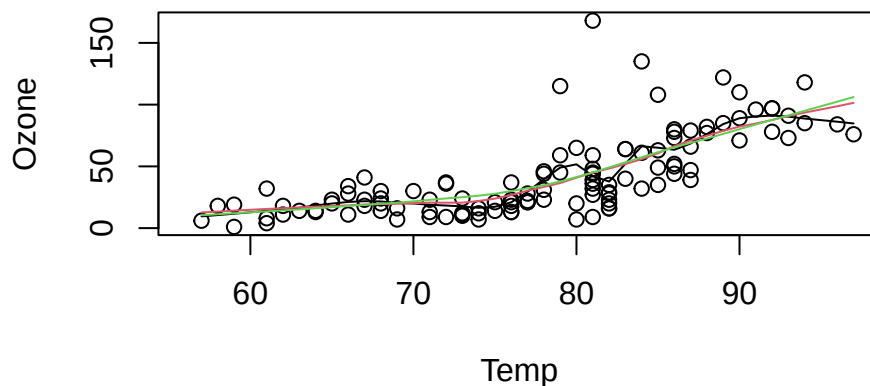


So, once again, we can try different spans.

```
lrfit1 <- loess(Ozone ~ Temp, data = airquality2, degree = 1, span = 0.2)
lrfit2 <- loess(Ozone ~ Temp, data = airquality2, degree = 1, span = 0.5)
lrfit3 <- loess(Ozone ~ Temp, data = airquality2, degree = 1, span = 0.8)
```

And the plot is:

```
with(airquality2, plot(Temp, Ozone))
lines(lrfit1$x, lrfit1$fitted) # black for 0.2
lines(lrfit2$x, lrfit2$fitted, col = 2) # red for 0.5
lines(lrfit3$x, lrfit3$fitted, col = 3) # green for 0.8
```



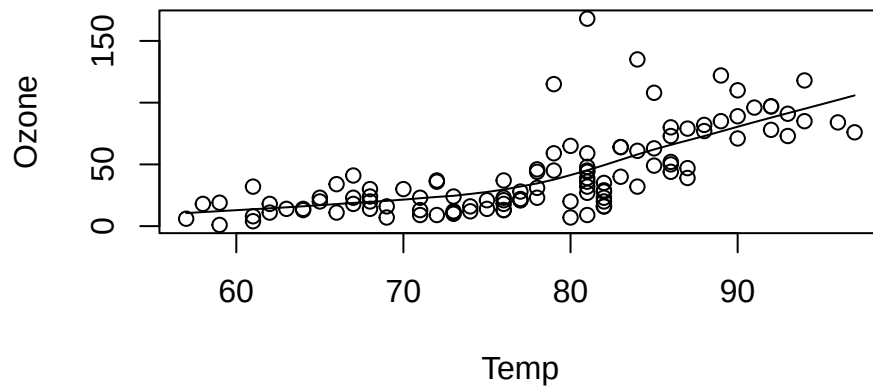
If we want R to select the span, we turn to the `loess.as` command. We need to remove missing values in the data here because the command doesn't process those out. (Again, you can think about how to improve on writing functions eventually for yourself, so this isn't an issue for users of functions you write.)

```
# note, this does complete cases over the ENTIRE data set,
# so it is not the best idea but it runs, how could you do better?
airquality2 <- airquality2[complete.cases(airquality2), ]
lrfit4 <- loess.as(airquality2$Temp, airquality2$Ozone, criterion = "gcv")
names(lrfit4) # objects in lrfit4 that we can access
```

```
## [1] "n"          "fitted"      "residuals"   "enp"         "s"           "one.delta"
## [7] "two.delta"  "trace.hat"   "divisor"     "robust"      "pars"        "kd"
## [13] "call"       "terms"      "xnames"      "x"           "y"           "weights"
```

The x and y -hat values are accessed the same way as before, so:

```
with(airquality2, plot(Temp, Ozone))
lines(lrfit4$x, lrfit4$fitted)
```

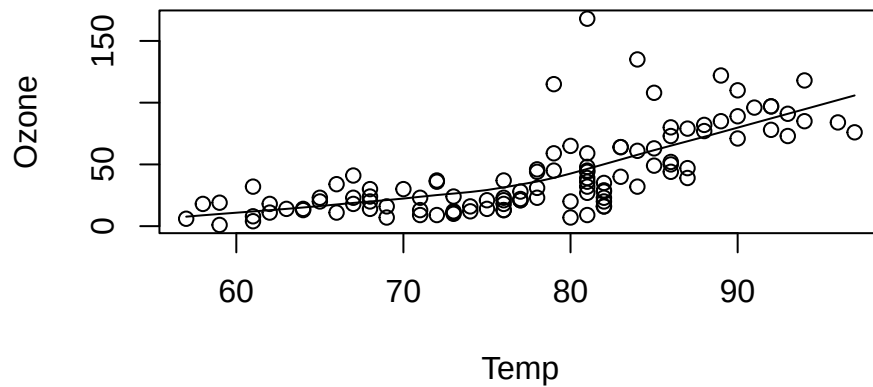


The other option you can try instead of *gcv* is *aicc*.

```
lrfit5 <- loess.as(airquality2$Temp, airquality2$Ozone, criterion = "aicc")
names(lrfit5) # objects in lrfit5 that we can access

## [1] "n"          "fitted"      "residuals"  "enp"        "s"          "one.delta"
## [7] "two.delta"  "trace.hat"  "divisor"    "robust"     "pars"       "kd"
## [13] "call"       "terms"      "xnames"     "x"          "y"          "weights"

with(airquality2, plot(Temp, Ozone))
lines(lrfit5$x, lrfit5$fitted)
```



If you wanted to see what spans these chose, you can get that in the following way:

```
(lrfit4$pars)$span

## [1] 0.7643601
```

```
(lrfit5$pars)$span # pulls span proportion out from parameters from model
```

```
## [1] 0.8892026
```

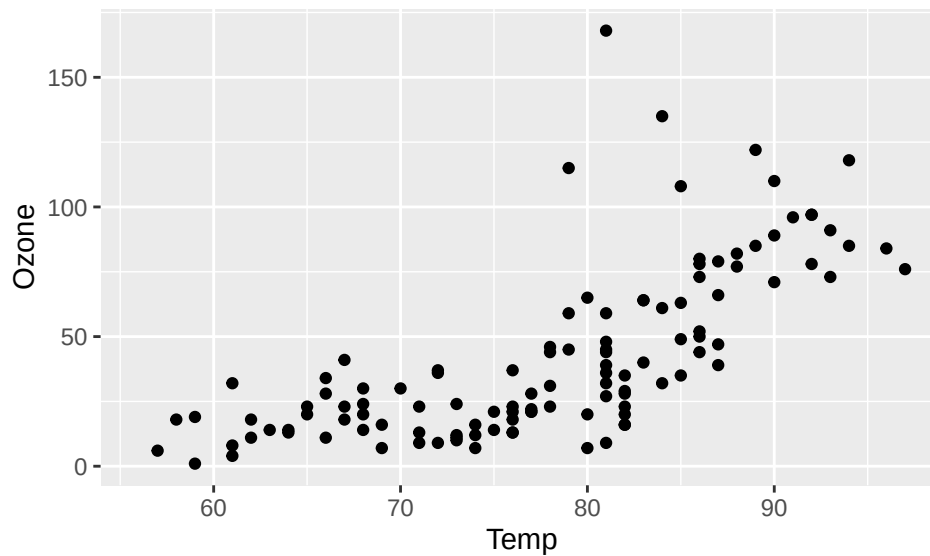
The default span in `loess` is 0.75, so these were picking values ABOVE that (using more points).

Examples in ggplot2

For those of you familiar with `ggplot2`, it has fairly easy ways to add smoothers to plots, including LOESS. We'll show some examples here, and try to explain some of the different options. If you aren't familiar with `ggplot2`, look for courses that use it at your institution.

First, we examine a basic scatterplot.

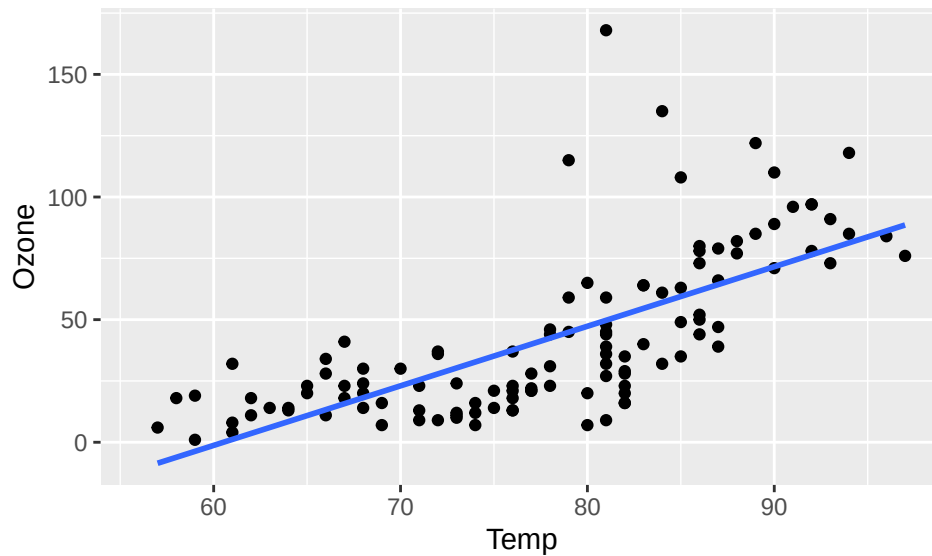
```
ggplot(data = airquality, aes(x = Temp, y = Ozone)) + geom_point()
```



Then, we consider adding lines to the plot to help capture the relationship. The next scatterplot has the parametric SLR line plotted.

```
ggplot(data = airquality, aes(x = Temp, y = Ozone)) +  
  geom_point() +  
  geom_smooth(method = "lm", se = FALSE)
```

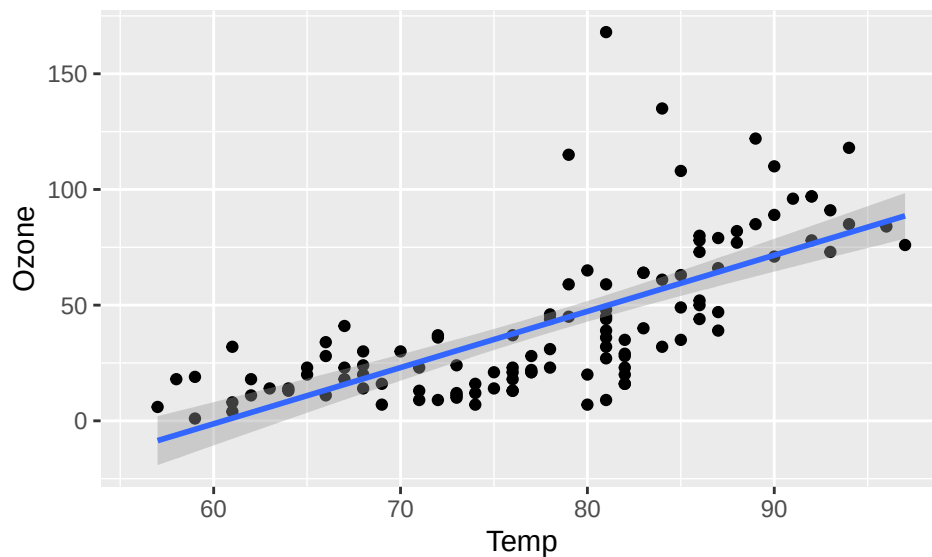
```
## `geom_smooth()` using formula = 'y ~ x'
```



Curious about what `se = FALSE` does? Set it to true. It turns out that true is also the default. This is giving you CIs.

```
ggplot(data = airquality, aes(x = Temp, y = Ozone)) +  
  geom_point() +  
  geom_smooth(method = "lm", se = TRUE)
```

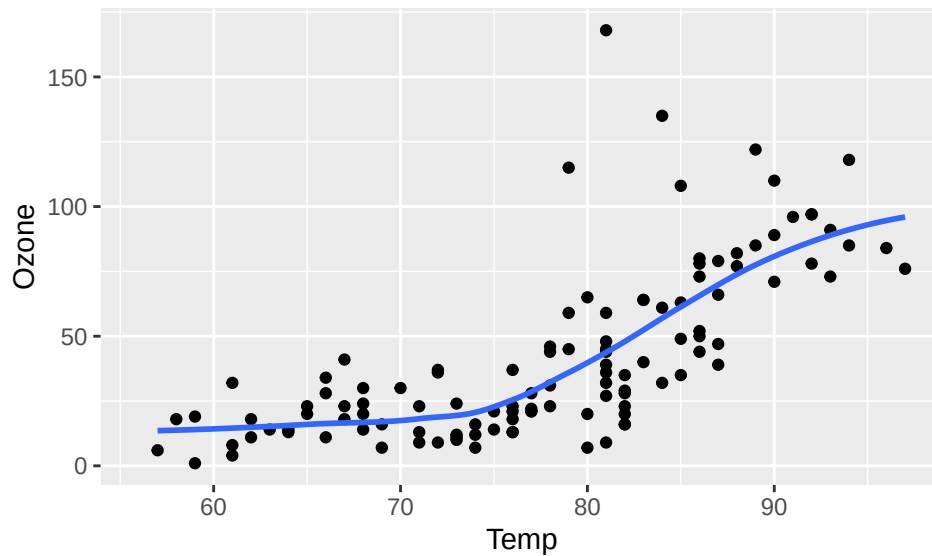
`geom\_smooth()` using formula = 'y ~ x'



Ok, but we want to see LOESS smoothers, so we need to change the smooth method. That's an easy change.

```
ggplot(data = airquality, aes(x = Temp, y = Ozone)) +  
  geom_point() +  
  geom_smooth(method = "loess", se = FALSE)
```

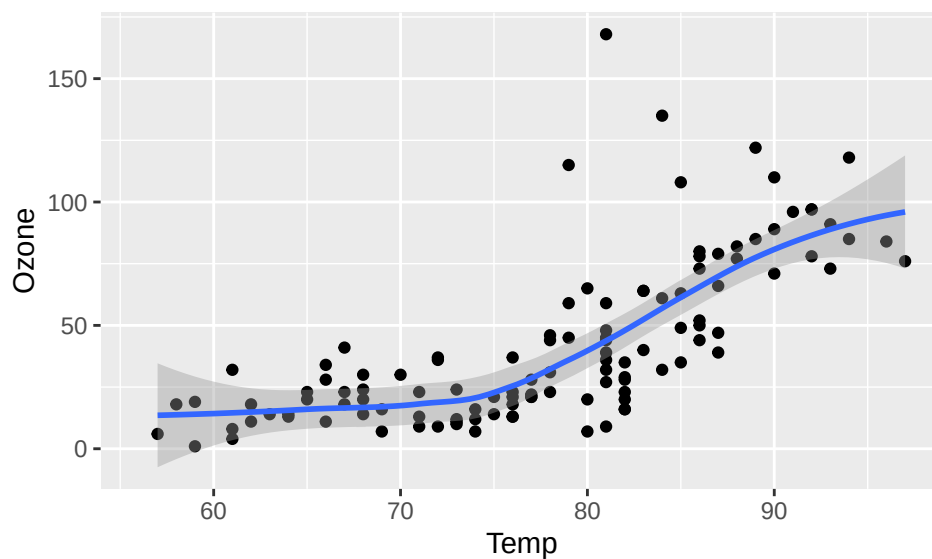
`geom\_smooth()` using formula = 'y ~ x'



Again with SE turned on that would be:

```
ggplot(data = airquality, aes(x = Temp, y = Ozone)) +  
  geom_point() +  
  geom_smooth(method = "loess", se = TRUE)
```

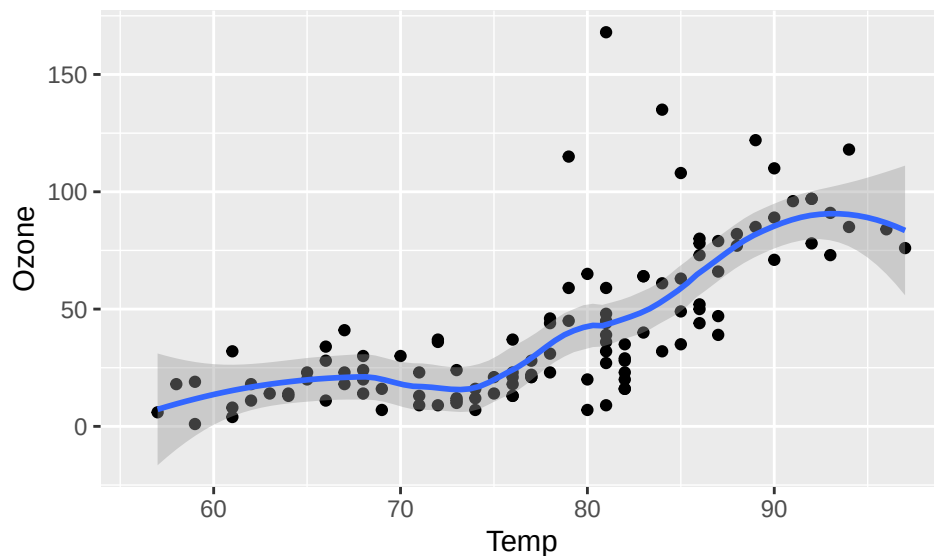
```
## `geom_smooth()` using formula = 'y ~ x'
```



This calls the loess function that we saw above, so it can handle any options you want to pass to it. The default span is 0.75 in loess, so what happens if we change that?

```
ggplot(data = airquality, aes(x = Temp, y = Ozone)) +  
  geom_point() +  
  geom_smooth(method = "loess", span = 0.5)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

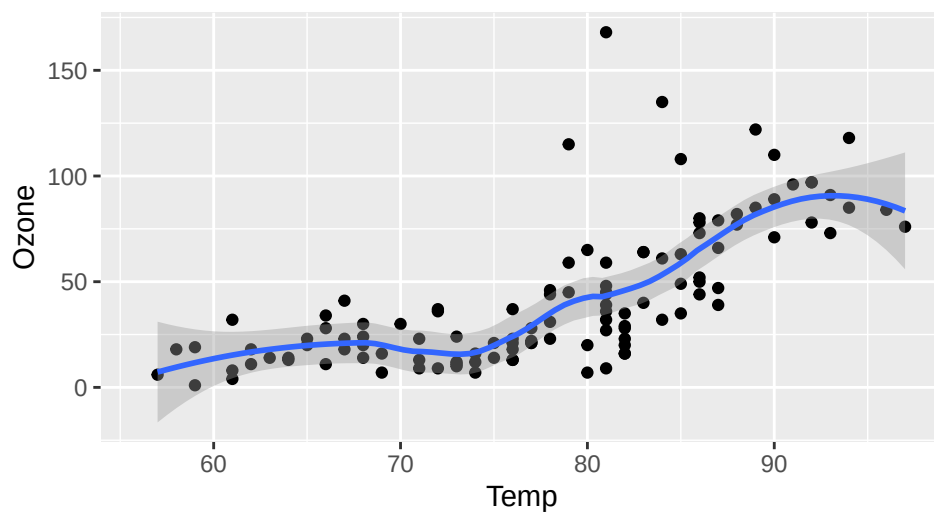



What if we wanted to add a title so we could tell the difference between the plots?

```
ggplot(data = airquality, aes(x = Temp, y = Ozone)) +
  geom_point() +
  geom_smooth(method = "loess", span = 0.5) +
  labs(title = "Loess Smoother with Span = 0.5")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Loess Smoother with Span = 0.5



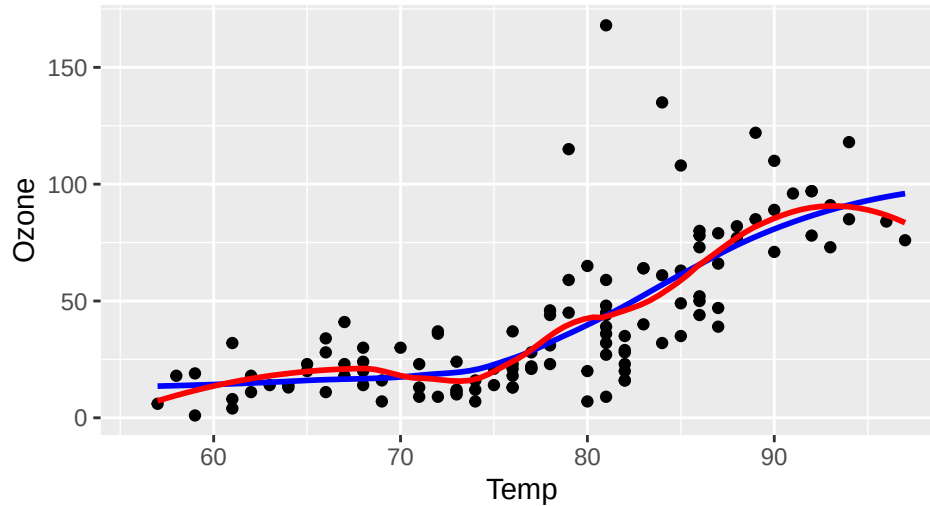
Or what if we wanted to plot both smoothers together?

```
ggplot(data = airquality, aes(x = Temp, y = Ozone)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, color = "blue") +
  geom_smooth(method = "loess", se = FALSE, span = 0.5, color = "red") +
  labs(title = "Loess Smoother with Span =0.5 (red) and 0.75 (blue)")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Loess Smoother with Span =0.5 (red) and 0.75 (blue)



Other options are available here. Feel free to play around with options during the lab. If you wanted to use `loess.as` to pick the span, run that separately and grab the span from the output as shown above, then feed that to the `loess` command here as the span.

Kernel Smoothing (Hollander, Wolfe, and Chicken 14.3)

The R command associated with kernel smoothing is `npreg` which is in the `np` library.

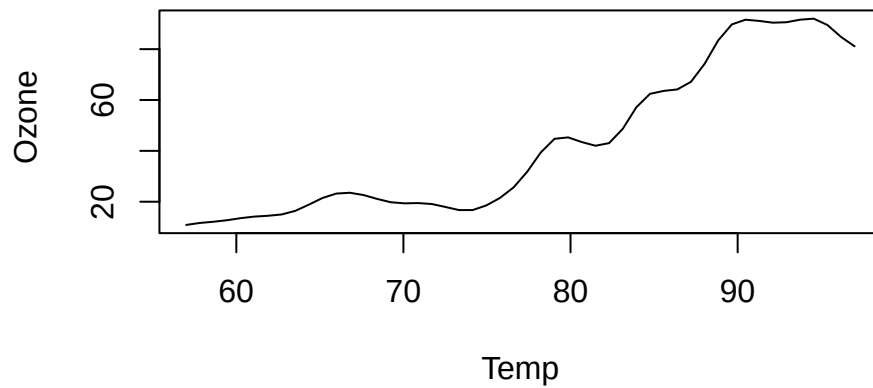
Our text states that the `npreg` command requires data sorted by X values, so we can use the `arrange` command (or the `sort` command, or several other options) to do this, just as we had to do for the LOESS method.

```
# to remind you of what we did above  
airquality2 <- arrange(airquality, Temp)
```

We run the basic command to get the default kernel smoothed estimate for $f(x)$ as follows:

```
npfit <- with(airquality2, npreg(Ozone ~ Temp))
```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multi.  
plot(npfit)
```

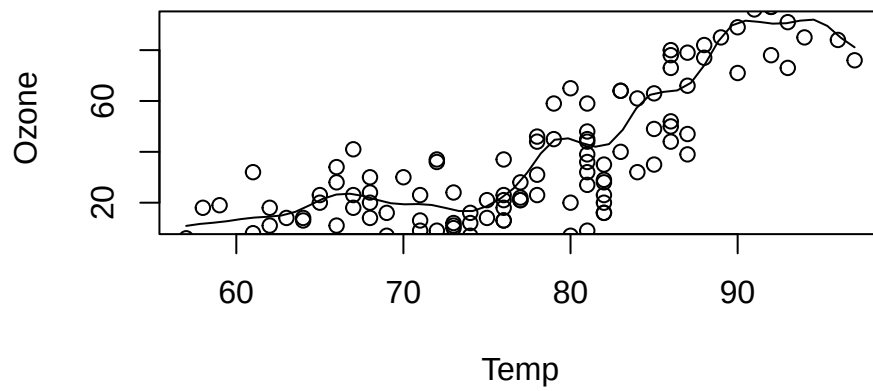


```
npfit # neat summary
```

```
##
## Regression Data: 116 training points, in 1 variable(s)
##           Temp
## Bandwidth(s): 1.305746
##
## Kernel Regression Estimator: Local-Constant
## Bandwidth Type: Fixed
##
## Continuous Kernel Type: Second-Order Gaussian
## No. Continuous Explanatory Vars.: 1
```

We need to plot this on top of our data, so we can just add the data points to the picture.

```
plot(npfit)
with(airquality2, points(Temp, Ozone))
```



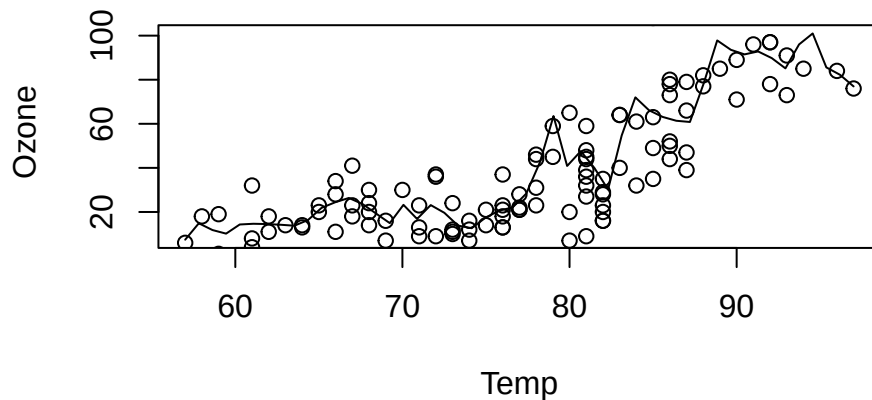
We can see what bandwidth was chosen as follows:

```
npfit$bw
```

```
## [1] 1.305746
```

The default kernel is Gaussian (normal), but you set the bandwidth with the `bw`s option.

```
npfit <- with(airquality2, npreg(Ozone ~ Temp, bw = 0.5))  
plot(npfit)  
with(airquality2, points(Temp, Ozone))
```



```
npfit$bw # verify
```

```
## [1] 0.5
```

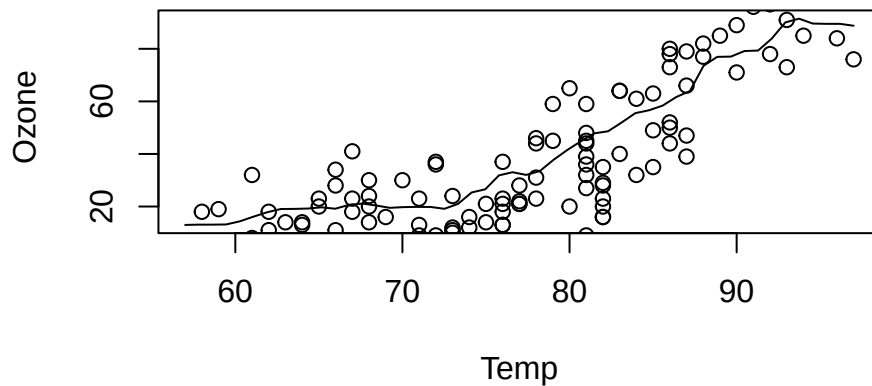
What if we want to change the kernel type? The options available are the gaussian, uniform (box), or epanechnikov via the `ckertype` option.

```
npfit2 <- with(airquality2, npreg(Ozone ~ Temp, ckertype = "uniform"))
```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multi
```

```
## Warning in rbandwidth(bw = tbw$bw, regtype = tbw$regtype, bwmethod =  
## tbw$method, : ignoring kernel order specified with uniform kernel type
```

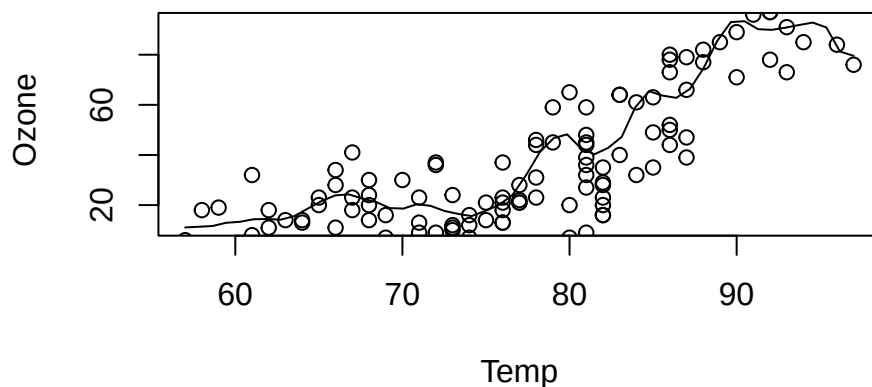
```
plot(npfit2)  
with(airquality2, points(Temp, Ozone))
```



This one gives a warning. You can get warnings because the bandwidth is not big enough to work well with the units of Temperature, or because the kernel order is too low to get a reasonable estimate. As we saw in kernel density estimation, the uniform kernel will give a more “blocky” estimate than the other kernels if the bandwidth is too low.

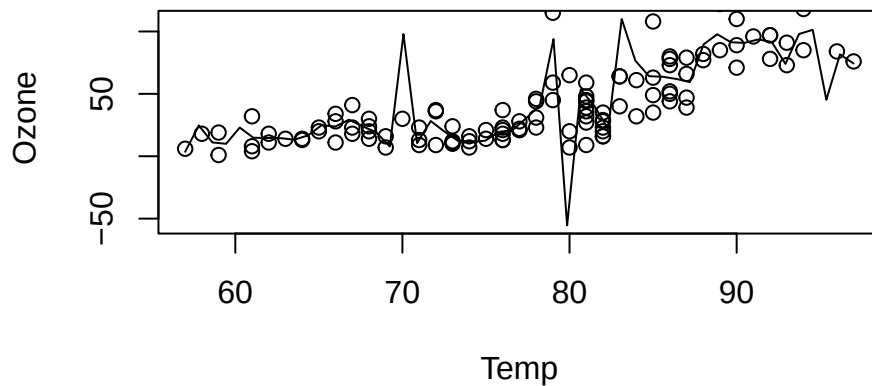
We can also change the order of the kernel. Let’s try the Epanechnikov kernel, with its default order of 2 and then with an order of 8. We’ll also set bandwidths of 1 for each to keep that consistent.

```
npfit3 <- with(airquality2,
  npreg(Ozone ~ Temp, bws = 1,
    ckertype = "epanechnikov"))
plot(npfit3)
with(airquality2, points(Temp, Ozone))
```



```
npfit4 <- with(airquality2,
  npreg(Ozone ~ Temp, bws = 1,
    ckertype = "epanechnikov",
    ckerorder = 8))
```

```
plot(npfit4)
with(airquality2, points(Temp, Ozone))
```



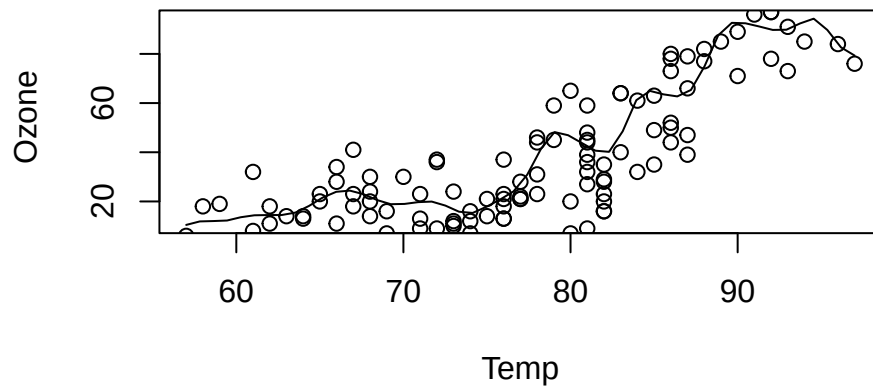
This order change is a big deal, and you can see the impact - especially in regions with less data. We would probably just keep the default order of 2. The change to higher order kernels can reduce bias, but adds variability (always a tradeoff).

What if we want R to help us find the best bandwidth or to re-specify the bandwidth as a fixed number of points to use? (That is, a span idea). For the re - specifying, we can do this with two methods - adaptive which changes the bandwidth at each data point, or generalized NN which can do it for any x. You can try those here.

```
npfit5 <- with(airquality2,
  npreg(Ozone ~ Temp,
    ckertype = "gaussian",
    bwtype = "adaptive_nn"))
```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multi
```

```
plot(npfit5)
with(airquality2, points(Temp, Ozone))
```

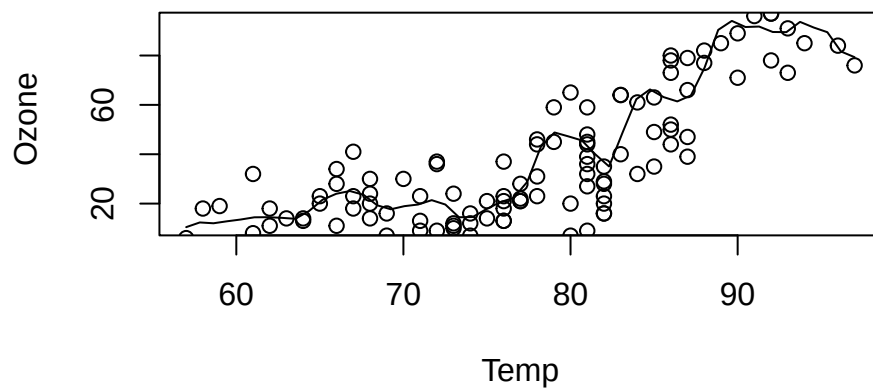


OR

```
npfit6 <- with(airquality2,
  npreg(Ozone ~ Temp,
    ckertype = "gaussian",
    bwtype = "generalized_nn"))
```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multi
```

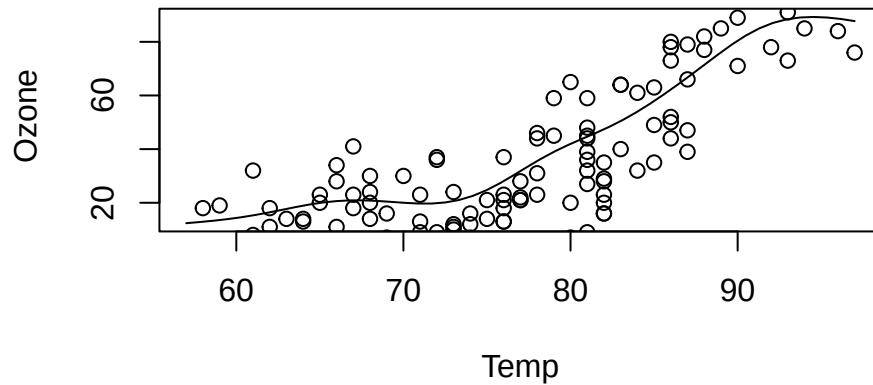
```
plot(npfit6)
with(airquality2, points(Temp, Ozone))
```



To get R to automatically pick the bandwidth using cross-validation, you have two choices:

```
npfit7 <- with(airquality2,
  npreg(Ozone ~ Temp,
    ckertype = "gaussian",
    bwmethod = "cv.aic"))
```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multi
plot(npfit7)
with(airquality2, points(Temp, Ozone))
```



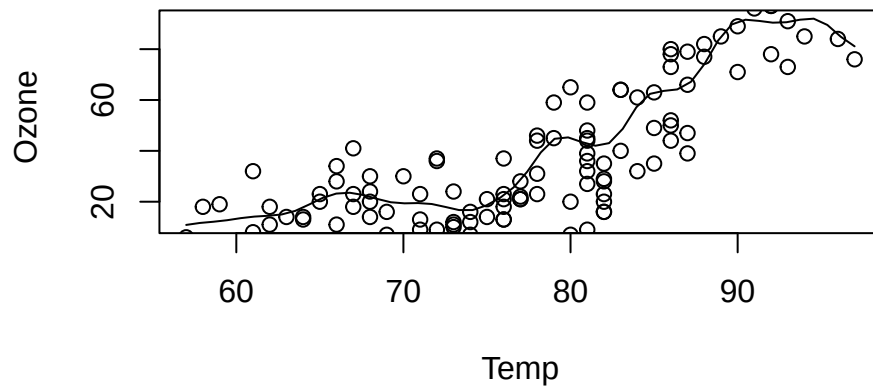
```
npfit7$bw
```

```
## [1] 2.916296
```

OR

```
npfit8 <- with(airquality2,
  npreg(Ozone ~ Temp,
    ckertype = "gaussian",
    bwmethod = "cv.ls"))
```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multi
plot(npfit8)
with(airquality2, points(Temp, Ozone))
```

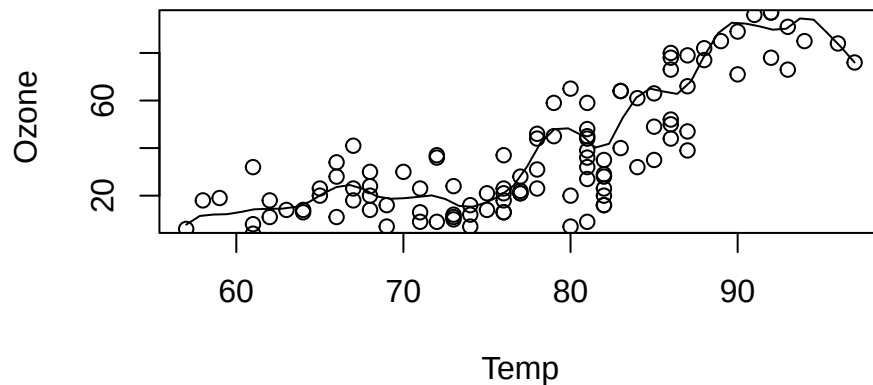
```
npfit8$bw
```

```
## [1] 1.305746
```

Note that these give very different results here - 2.92 and 1.31. The 1.31 is the default method used above (if you go back to check `npfit`, that's what we got).

Finally, you can try local linear kernel regression (using the kernels as weights in the local regression method) with:

```
# can change bws, etc.
npfit9 <- with(airquality2,
  npreg(Ozone ~ Temp, bws = 1,
    ckertype = "gaussian",
    regtype = "ll"))
plot(npfit9)
with(airquality2, points(Temp, Ozone))
```



Have fun as you explore the wide variety of different options for smoothing.

Data Sources

Airquality data. Available in R with `data(airquality)`. Help references Chambers, J. M., Cleveland, W. S., Kleiner, B. and Tukey, P. A. (1983) Graphical Methods for Data Analysis. Belmont, CA: Wadsworth.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.